TUGAS BESAR

IF2224 - Teori Bahasa Formal dan Otomata

Milestone 3

SEMANTIC ANALYSIS

DongunGanteng

Benedictus Nelson - 13523150

Rainaldi Pratama F. Sembiring - 13524117

Jonathan Alveraldo Bangun - 13524120

Ramadhian Nabil Firdaus Gumay - 13524126

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

DAFTAR ISI

1. Landasan Teori

1.1. *Semantic Analysis*

Dalam proses kompilasi, setelah tahap *lexical analysis* menghasilkan token dan *syntax analysis* menghasilkan parse tree, masih ada kemungkinan bahwa program yang ditulis tidak bermakna secara logika. Contohnya, pernyataan x := x + true bisa saja lolos dari pengecekan sintaks karena strukturnya benar secara gramatikal, tetapi secara semantik tidak valid karena tidak bisa menjumlahkan integer dengan boolean. Di sinilah peran *Semantic Analysis*.

Semantic Analysis adalah tahap kompilasi yang bertugas memeriksa makna dari program, bukan sekedar strukturnya. Secara umum, ada empat hal yang diperiksa pada tahap ini:

i. *Type Checking*, memastikan setiap operator digunakan dengan operand yang memiliki tipe data kompatibel. Misalnya, operator aritmatika hanya boleh digunakan pada nilai Integer atau Real, dan operator logika *and/or/not* hanya bolehh digunakan pada nilai Boolean.

ii. *Symbol Table Management*, memastikan setiap identifier sudah dideklarasikan sebelum digunakan dan tidak dideklarasikan ulang dalam scope yang sama. Semua informasi identifier disimpan dalam struktur data yang disebut *symbol table*.

iii. *Scope Resolution*, menentukan identifier mana yang valid untuk diakses dari suatu titik tertentu dalam program, berdasarkan hierarki blok kode. Variabel yang dideklarasikan di dalam suatu prosedur tidak bisa diakses dari luar prosedur tersebut.

iv. *Control Flow Validation*, memastikan alur kontrol program logis, seperti kondisi pada *if-statement* dan *while-statement* harus bertipe Boolean, dan setiap fungsi memiliki nilai kembalian yang sesuai dengan tipe yang dideklarasikan.

Hasil dari *Semantic Analysis* adalah parse tree yang sudah dianotasi dengan informasi semantik, yaitu *Decorated Abstract Syntax Tree* beserta *symbol table* yang terisi lengkap.

1.2. *Abstract Syntax Tree*

Parse tree yang dihasilkan oleh parser masih mengandung banyak informasi sintaktis yang tidak lagi diperlukan pada tahap selanjutnya, seperti tanda kurung, keyword *begin*, *end*, dan titik koma. Oleh karena itu, sebelum dilakukan analisis semantik, parse tree perlu dikonversi menjadi bentuk yang lebih ringkas yang disebut *Abstract Syntax Tree (AST*).

AST adalah representasi pohon dari struktur program yang hanya menyimpan informasi yang relevan secara semantik. Setiap node internal merepresentasikan sebuah konstruksi dalam

bahasa pemrograman, dan anak-anaknya merepresentasikan komponen dari konstruksi tersebut. Perbedaan utama AST dengan parse tree adalah pada AST, node-node perantara yang hanya ada karena kebutuhan grammar (seperti *expression*, *term*, *factor*) sudah dihilangkan dan hanya tersisa hubungan semantik antar operator dan operand.

Sebagai contoh, untuk ekspresi *a + b * c*, parse tree akan memiliki banyak node perantara yang merepresentasikan aturan grammar. Sedangkan pada AST, representasinya lebih ringkas: sebuah node *BinOp(+)* dengan anak kiri *Var(a)* dan anak kanan *BinOp(*) [Var(b), Var(c)]*. Proses konversi dari parse tree ke AST ini dilakukan menggunakan teknik yang disebut *Syntax-Directed Translation (SDT)*, di mana semantic actions disisipkan ke dalam aturan produksi grammar untuk membangun node-node AST secara bertahap.

1.3. *Decorated Abstract Syntax Tree*

Setelah AST terbentuk, Semantic Analysis melakukan proses *dekorasi* atau anotasi terhadap setiap node di dalamnya. Proses ini menghasilkan apa yang disebut sebagai *Decorated AST*. Informasi yang ditambahkan pada setiap node antara lain:

v. Tipe data akhir dari ekspresi yang direpresentasikan oleh node tersebut, misalnya node BinOp(+) antara dua Integer akan dianotasi dengan tipe Integer.

vi. Referensi atau indeks ke entri yang bersangkutan di symbol table, sehingga informasi lengkap tentang identifier bisa langsung diakses.

vii. Tingkatan lexical level atau scope tempat node tersebut berada.

Decorated AST inilah yang menjadi output utama dari fase *Semantic Analysis* dan menjadi input untuk fase kompilasi berikutnya.

1.4. *Attributed Grammar* dan *L-Attributed Grammar*

Untuk mengimplementasikan Semantic Analysis secara sistematis, digunakan pendekatan yang disebut *Attributed Grammar*. Attributed Grammar adalah perluasan dari context-free grammar yang dilengkapi dengan atribut dan aturan semantik pada setiap produksi. Atribut adalah nilai-nilai yang diasosiasikan dengan simbol-simbol dalam grammar, dan aturan semantik menjelaskan bagaimana nilai atribut tersebut dihitung.

Terdapat dua jenis atribut, yaitu *synthesized attributes*, nilainya dihitung dari atribut anak-anaknya dalam parse tree (bottom-up) dan *inherited attributes*, nilainya dihitung dari atribut parent atau saudaranya (top-down). Pada implementasi ini digunakan pendekatan *L-Attributed Grammar*, yaitu grammar di mana setiap inherited attribute dari suatu simbol

hanya bergantung pada atribut simbol-simbol yang ada di sebelah kirinya dalam aturan produksi yang sama. L-Attributed Grammar dipilih karena bisa dievaluasi secara efisien dalam satu kali penelusuran dari kiri ke kanan (*single-pass left-to-right*), yang cocok untuk implementasi berbasis *recursive descent*.

1.5. *Symbol Table*

Symbol table adalah struktur data sentral dalam proses kompilasi yang digunakan untuk menyimpan informasi mengenai setiap identifier yang ditemukan dalam source code. Informasi yang disimpan biasanya meliputi nama identifier, jenis objeknya (variabel, konstanta, fungsi, prosedur, tipe), tipe datanya, lexical level tempat identifier dideklarasikan, dan alamat memorinya.

Secara konseptual, symbol table menggunakan struktur data *stack* untuk mendukung pengelolaan scope secara bertingkat. Ketika program memasuki blok baru (misalnya prosedur atau fungsi), scope baru di-push ke stack. Ketika keluar, scope tersebut di-pop. Pencarian identifier (*lookup*) dilakukan dari scope terdalam ke scope terluar, sehingga variabel lokal akan menutupi (*shadow*) variabel dengan nama yang sama di scope yang lebih luar.

Pada implementasi Semantic Analysis untuk bahasa Arion, symbol table diorganisasi ke dalam tiga tabel yang saling terintegrasi, yaitu *tab* sebagai tabel utama yang menyimpan semua identifier beserta atributnya, *btab* yang menyimpan informasi blok prosedur/fungsi, dan *atab* yang menyimpan informasi spesifik untuk tipe array. Ketiga tabel ini bekerja bersama untuk merepresentasikan struktur scope dan tipe data program secara lengkap.

1.6. *Type Compatibility*

Type Compatibility adalah aturan yang menentukan kapan dua buah tipe data dianggap kompatibel untuk digunakan bersama. Dalam bahasa yang berbasis Pascal seperti Arion, terdapat dua konsep utama yang perlu dibedakan:

Pertama adalah *type compatibility*, yaitu kondisi di mana dua tipe bisa digunakan secara bersamaan misalnya dalam operasi perbandingan. Dua tipe dikatakan compatible apabila keduanya memiliki tipe dasar yang sama, salah satunya merupakan subrange dari yang lain, atau keduanya subrange dari tipe dasar yang sama.

Kedua adalah *assignment compatibility*, yaitu aturan yang lebih ketat yang berlaku khusus untuk pernyataan assignment (:=). Salah satu aturan penting di sini adalah *widening conversion*, yaitu nilai bertipe Integer boleh di-assign ke variabel bertipe Real, tetapi tidak

sebaliknya. Selain itu, terdapat aturan bahwa dua anonymous record, meskipun memiliki struktur yang sama persis, tetap dianggap incompatible dalam assignment.

2. Perancangan Dan Implementasi

2.1. Teknologi dan Struktur Program

Program *Semantic Analyzer* ini diimplementasikan menggunakan bahasa C++. Kode dibagi secara modular berdasarkan tanggung jawabnya menggunakan prinsip pemrograman berorientasi objek (OOP):

a. ASTBuilder (ast_builder.h, ast_builder.cpp): Kelas yang bertugas melakukan *Syntax-Directed Translation*. Kelas ini menerima *Parse Tree* dan menelusurinya secara dinamis untuk membentuk objek turunan dari ASTNode (seperti ProgramNode, BinaryOpNode, AssignStmtNode, dll.), menghilangkan terminal yang tidak berguna (seperti *keyword* begin, end, ;).

   Entry point ASTBuilder::build() memetakan tiga cabang utama parse tree (<program-header>, <declaration-part>, <compound-statement>) menjadi field ProgramNode:

```
ProgramNode* ASTBuilder::build(ParseTreeNode* root) {
    if (!root || root->label != "<program>") return nullptr;
    ProgramNode* progNode = new ProgramNode();

    for (auto child : root->children) {
        if (child->label == "<program-header>") {
            if (child->children.size() > 1)
                progNode->name = child->children[1]->value;
        }
        else if (child->label == "<declaration-part>") {
            buildDeclarations(child, progNode->declarations);
        }
        else if (child->label == "<compound-statement>") {
            progNode->mainBlock = buildCompoundStatement(child);
        }
    }
    return progNode;
}
```

b. SemanticVisitor & SemanticAnalyzer (semantic.h, semantic_decl.cpp, semantic_expr.cpp, semantic_stmt.cpp): Menerapkan pola desain *Visitor Pattern* (*L-Attributed Grammar*) untuk menelusuri AST secara *Depth-First Search* (DFS). Kelas ini mengevaluasi setiap *node*, melakukan resolusi tipe data, dan mencatat informasi ke dalam tabel simbol. Pengerjaan dibagi secara terstruktur: Modul Deklarasi, Modul Ekspresi (oleh Rama), dan Modul Statement (oleh Nelson).

   Setiap subclass ASTNode mengimplementasikan accept() yang melakukan *double-dispatch* ke metode visit() yang tepat:

```
class ASTNode {
public:
    DataType evalType;
    int symRef;
    int lexicalLevel;
    virtual void accept(SemanticVisitor* visitor) = 0;
};

void BinaryOpNode::accept(SemanticVisitor* visitor) {
visitor->visit(this); }
void AssignStmtNode::accept(SemanticVisitor* visitor) {
visitor->visit(this); }
```

c. SymbolTable: Struktur data pengelola lingkungan *(environment)* yang menyimpan deklarasi *identifier*. Menggunakan tiga tabel utama yang terintegrasi:

   i. tab: Tabel utama penyimpan *identifier* beserta jenis objek, tipe data dasar, tingkatan level leksikal, dan alamat memori.

   ii. atab: Tabel penyimpan informasi spesifik array (batas bawah, atas, tipe indeks, dan elemen).

   iii. btab: Tabel pengelola *block scope* (prosedur, fungsi) untuk memetakan hierarki dan memori.

   Definisi ketiga struct ini di semantic.h:

```
struct TabEntry {
    string identifiers;
    int link;
    ObjClass obj;
    DataType type;
```

```
      int ref;
      int nrm, lev, adr;
  };
  struct ATabEntry { int arrays; DataType xtyp, etyp; int eref, low,
high, elsz, size; };
  struct BTabEntry { int blocks, last, lpar, psze, vsze; };
```

Field link pada TabEntry adalah kunci teknik *threaded scope*: setiap identifier menyimpan indeks identifier sebelumnya di scope yang sama, sehingga *lookup* dapat menelusuri rantai indeks tanpa scanning linier.

2.2. Alur Kerja (Work Flow)

a. Rekonstruksi Tree / Tokenisasi: Pada main.cpp, sistem secara otomatis mendeteksi apakah input berupa daftar token, *source code* utuh, atau format pohon sintaksis (*Parse Tree*).

b. Pembangunan AST: ASTBuilder::build() dipanggil. Evaluasi dilakukan secara bottom-up untuk node-node dasar pembentuk ekspresi, dan top-down untuk blok struktur utama (*Program*, *Block*).

c. Inisialisasi Lingkungan: SemanticAnalyzer membuat lingkungan pradefinisi (*Predefined Identifiers*) seperti *Integer*, *Real*, *True*, dan *False* pada leksikal level 0. Implementasinya mengisi 33 reserved words (indeks 0–32) dan 9 predefined identifier (indeks 33–41) supaya identifier user tidak bisa shadow keyword:

```
   void SymbolTable::initPredefined() {
       string reservedWords[33] = {

"empty","and","array","begin","case","const","div","downto",
           "do","else","end","for","function","if","mod","not","of",

"or","procedure","program","record","repeat","integer","real",

"boolean","char","string","then","to","type","until","var","while"
       };
       for (int i = 0; i <= 32; i++) { /* push entri dummy const */
}
```

```
        insertTab("integer", ObjClass::TYPE_DEF, DataType::INTEGER,
0, 0, 0);
        insertTab("real",    ObjClass::TYPE_DEF, DataType::REAL,
0, 0, 0);
        insertTab("char",    ObjClass::TYPE_DEF, DataType::CHAR,
0, 0, 0);
        insertTab("boolean", ObjClass::TYPE_DEF, DataType::BOOLEAN,
0, 0, 0);
        insertTab("string",  ObjClass::TYPE_DEF, DataType::STRING,
0, 0, 0);
        insertTab("true",    ObjClass::CONSTANT,  DataType::BOOLEAN,
0, 0, 1);
        insertTab("false",   ObjClass::CONSTANT,  DataType::BOOLEAN,
0, 0, 0);
        insertTab("writeln", ObjClass::PROCEDURE, DataType::NONE,
0, 0, 0);
        insertTab("readln",  ObjClass::PROCEDURE, DataType::NONE,
0, 0, 0);

        activeBlocks.push_back(insertBTab());
    }
```

Inilah alasan output tab: pada bagian Pengujian selalu diawali baris 0–41 yang identik di seluruh kasus uji.

d. Dekorasi AST (Type & Scope Checking): Melalui *Visitor*, kode berjalan memanggil accept() pada setiap node.

    i. Ketika memasuki blok deklarasi prosedur/fungsi (SubprogDeclNode), *Scope* baru dibuka (pushScope), dan ditutup (popScope) ketika keluar. lookupTab() menelusuri scope dari yang terdalam ke terluar dengan mengikuti rantai link sehingga variabel lokal otomatis menutupi *shadow* variabel global bernama sama:

```
    TabEntry* SymbolTable::lookupTab(string name) {
        for (int i = activeBlocks.size() - 1; i >= 0; i--) {
            int currIdx = btab[activeBlocks[i]].last;
            while (currIdx > 0 && currIdx < tab.size()) {
                if (tab[currIdx].identifiers == name) return
&tab[currIdx];
                currIdx = tab[currIdx].link;
            }
```

```
        }
        for (int i = 32; i >= 0; i--)
            if (tab[i].identifiers == name) return &tab[i];
        return nullptr;
    }
```

ii. Tipe data dievaluasi secara berjenjang dari daun (*LiteralNode*, *VarAccessNode*) ke atas (misalnya BinaryOpNode). Pola ini bersifat *bottom-up* sehingga setiap parent node bisa membaca evalType anak yang sudah terisi:

```
    void SemanticAnalyzer::visit(BinaryOpNode* node) {
        node->left->accept(this);
        node->right->accept(this);
        DataType lType = node->left->evalType;
        DataType rType = node->right->evalType
    }
```

2.3. Type Compatibility & Semantic Rules

Aturan kesesuaian tipe *(Type Compatibility)* diimplementasikan sesuai kebutuhan bahasa Arion:

Assignment Compatibility: Berdasarkan metode isAssignmentCompatible(), variabel tujuan dan nilai ekspresi harus memiliki tipe data yang sama. Namun, mendukung aturan *widening* di mana nilai bertipe Integer diizinkan masuk ke penampung Real. Implementasi lengkapnya di semantic_stmt.cpp:

```
static bool isAssignmentCompatible(DataType target, DataType value)
{
    if (target == DataType::NOTYPE || value == DataType::NOTYPE)
return true;
    if (target == value) return true;
    if (target == DataType::REAL && value == DataType::INTEGER)
return true;
    if ((target == DataType::SUBRANGE && value == DataType::INTEGER)
||
        (target == DataType::INTEGER && value == DataType::SUBRANGE)
||
```

```
        (target == DataType::SUBRANGE && value ==
DataType::SUBRANGE))
        return true;
// subrange
    return false;
}
```

Aritmatika: Operator (+, -, *) mendukung kombinasi Integer dan Real. Jika salah satu operand adalah Real, maka evalType akan otomatis menjadi Real. Pembagian khusus Real (/) akan selalu menghasilkan Real, sedangkan div dan mod secara eksklusif mensyaratkan kedua *operand* bertipe Integer.

```
if (op == "plus" || op == "minus" || op == "times") {
    if ((lType == DataType::INTEGER || lType == DataType::REAL) &&
        (rType == DataType::INTEGER || rType == DataType::REAL)) {
        node->evalType = (lType == DataType::REAL || rType ==
DataType::REAL)
                        ? DataType::REAL : DataType::INTEGER;
    } else {
        cout << "Error Semantik: Tipe data tidak cocok untuk operasi
aritmatika!"
             << endl;
        node->evalType = DataType::NOTYPE;
    }
}
else if (op == "idiv" || op == "imod") {                    if (lType
== DataType::INTEGER && rType == DataType::INTEGER)
        node->evalType = DataType::INTEGER;
}
else if (op == "rdiv") {
    node->evalType = DataType::REAL;
}
```

Relational & Logika: Operator perbandingan (=, <>, <, >, dll.) mewajibkan kedua operand memiliki tipe yang komparabel (sesama *Integer/Real* atau sesama *String/Char*) dan akan dianotasi dengan *return type* Boolean. Statement kondisi pada *If* dan *While* diwajibkan memiliki evaluasi kondisi Boolean. Toleransi *NOTYPE* diberikan agar tidak terjadi *cascade error* setelah ekspresi yang sudah dilaporkan salah:

```
void SemanticAnalyzer::visit(IfStmtNode* node) {
    if (node->condition) {
        node->condition->accept(this);
        if (node->condition->evalType != DataType::BOOLEAN &&
            node->condition->evalType != DataType::NOTYPE)
            cerr << "Semantic Error: 'if' condition must be Boolean
(got "
                 << dataTypeName(node->condition->evalType) << ")."
<< endl;
    }
    if (node->thenStmt) node->thenStmt->accept(this);
    if (node->elseStmt) node->elseStmt->accept(this);
    node->evalType = DataType::NONE;
}
```

ForStmtNode menambahkan aturan ekstra: iterator wajib bertipe *ordinal* (Integer/Char/Boolean) , dan ekspresi *start/end* harus *assignment-compatible* dengan iterator.

3. Pengujian

<table>
<tr><th>No</th><th>Input</th></tr>
<tr><td rowspan="3">1</td><td>

```
program Hello;

var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.
```

</td></tr>
<tr><th>Output</th></tr>
<tr><td>

```
tab:
idx  id          obj       type  ref  nrm  lev  adr  link
-------------------------------------------------------
0    empty       const     1     0    0    0    0    0
1    and         const     1     0    0    0    0    0
```

</td></tr>
</table>

2	array	const	1	0	0	0	0	0
3	begin	const	1	0	0	0	0	0
4	case	const	1	0	0	0	0	0
5	const	const	1	0	0	0	0	0
6	div	const	1	0	0	0	0	0
7	downto	const	1	0	0	0	0	0
8	do	const	1	0	0	0	0	0
9	else	const	1	0	0	0	0	0
10	end	const	1	0	0	0	0	0
11	for	const	1	0	0	0	0	0
12	function	const	1	0	0	0	0	0
13	if	const	1	0	0	0	0	0
14	mod	const	1	0	0	0	0	0
15	not	const	1	0	0	0	0	0
16	of	const	1	0	0	0	0	0
17	or	const	1	0	0	0	0	0
18	procedure	const	1	0	0	0	0	0
19	program	program	1	0	0	0	0	0
20	record	const	1	0	0	0	0	0
21	repeat	const	1	0	0	0	0	0
22	integer	const	1	0	0	0	0	0
23	real	const	1	0	0	0	0	0
24	boolean	const	1	0	0	0	0	0
25	char	const	1	0	0	0	0	0
26	string	const	1	0	0	0	0	0
27	then	const	1	0	0	0	0	0
28	to	const	1	0	0	0	0	0
29	type	const	1	0	0	0	0	0
30	until	const	1	0	0	0	0	0
31	var	const	1	0	0	0	0	0
32	while	const	1	0	0	0	0	0
33	integer	type	2	0	0	0	0	0
34	real	type	3	0	0	0	0	33
35	char	type	4	0	0	0	0	34
36	boolean	type	5	0	0	0	0	35
37	string	type	6	0	0	0	0	36
38	true	const	5	0	1	0	1	37
39	false	const	5	0	1	0	0	38
40	writeln	procedure	1	0	0	0	0	39
41	readln	procedure	1	0	0	0	0	40
42	Hello	program	1	0	0	0	0	41
43	a	variable	2	0	1	0	0	42

```
44   b           variable  2     0    1    0    0    43

btab:
idx  last  lpar  psze  vsze
--------------------------
0    44    0     0     2    <- global block (program), vsze=2
1    44    0     0     0    <- main block (kosong variabel lokal)

atab: (kosong karena tidak ada array)

Decorated AST:
ProgramNode(name: 'Hello')
 ├─ Declarations
 │    ├─ VarDecl('a')          → tab_index:43, type:unknown, lev:0
 │    └─ VarDecl('b')          → tab_index:44, type:unknown, lev:0
 └─ Block                    → block_index:42, lev:0
     ├─ Assign('a' := 5)      → type:void
     │    ├─ target 'a'       → tab_index:42, type:integer
     │    └─ value 5          → type:integer
     ├─ Assign('b' := a + 10)      → type:void
     │    ├─ target 'b'       → tab_index:43, type:integer
     │    └─ value BinOp 'plus'           → type:integer
     │        ├─ 'a'          → tab_index:42, type:integer
     │        └─ 10           → type:integer
     └─ writeln(...)          → predefined, tab_index:-1
         ├─ arg 'Result = '        → type:string
         └─ arg 'b'           → tab_index:43, type:integer
```

Screenshot Output

test > milestone-3 > output-1.txt

```
tab:
idx  id          obj       type  ref  nrm  lev  adr  link
-------------------------------------------------------
0    empty       const     1     0    0    0    0    0
1    and         const     1     0    0    0    0    0
2    array       const     1     0    0    0    0    0
3    begin       const     1     0    0    0    0    0
4    case        const     1     0    0    0    0    0
5    const       const     1     0    0    0    0    0
6    div         const     1     0    0    0    0    0
7    downto      const     1     0    0    0    0    0
8    do          const     1     0    0    0    0    0
9    else        const     1     0    0    0    0    0
10   end         const     1     0    0    0    0    0
11   for         const     1     0    0    0    0    0
12   function    const     1     0    0    0    0    0
13   if          const     1     0    0    0    0    0
14   mod         const     1     0    0    0    0    0
15   not         const     1     0    0    0    0    0
16   of          const     1     0    0    0    0    0
17   or          const     1     0    0    0    0    0
18   procedure   const     1     0    0    0    0    0
19   program     program   1     0    0    0    0    0
20   record      const     1     0    0    0    0    0
21   repeat      const     1     0    0    0    0    0
22   integer     const     1     0    0    0    0    0
23   real        const     1     0    0    0    0    0
24   boolean     const     1     0    0    0    0    0
25   char        const     1     0    0    0    0    0
26   string      const     1     0    0    0    0    0
27   then        const     1     0    0    0    0    0
28   to          const     1     0    0    0    0    0
29   type        const     1     0    0    0    0    0
30   until       const     1     0    0    0    0    0
31   var         const     1     0    0    0    0    0
32   while       const     1     0    0    0    0    0
33   integer     type      2     0    0    0    0    0
34   real        type      3     0    0    0    0    33
35   char        type      4     0    0    0    0    34
```

test > milestone-3 > output-1.txt

```
39   35   char        type      4     0    0    0    0    34
40   36   boolean     type      5     0    0    0    0    35
41   37   string      type      6     0    0    0    0    36
42   38   true        const     5     0    1    0    1    37
43   39   false       const     5     0    1    0    0    38
44   40   writeln     procedure 1     0    0    0    0    39
45   41   readln      procedure 1     0    0    0    0    40
46   42   Hello       program   1     0    0    0    0    41
47   43   a           variable  2     0    1    0    0    42
48   44   b           variable  2     0    1    0    0    43
49
50   btab:
51   idx  last  lpar  psze  vsze
52   ---------------------------
53   0    44    0     0     2    <- global block (program), vsze=2
54   1    44    0     0     0    <- main block (kosong variabel lokal)
55
56   atab: (kosong karena tidak ada array)
57
58   Decorated AST:
59   ProgramNode(name: 'Hello')
60    ├─ Declarations
61    │    ├─ VarDecl('a')       → tab_index:43, type:unknown, lev:0
62    │    └─ VarDecl('b')       → tab_index:44, type:unknown, lev:0
63    └─ Block                  → block_index:42, lev:0
64         ├─ Assign('a' := 5)    → type:void
65         │    ├─ target 'a'      → tab_index:42, type:integer
66         │    └─ value 5         → type:integer
67         ├─ Assign('b' := a + 10)   → type:void
68         │    ├─ target 'b'      → tab_index:43, type:integer
69         │    └─ value BinOp 'plus'      → type:integer
70         │         ├─ 'a'         → tab_index:42, type:integer
71         │         └─ 10      → type:integer
72         └─ writeln(...)        → predefined, tab_index:-1
73              ├─ arg 'Result = '         → type:string
74              └─ arg 'b'         → tab_index:43, type:integer
```

2	Input

```
program HitungLuas;

var
  panjang, lebar, luas: integer;
```

```
begin
  panjang := 15;
  lebar := 8;
  luas := panjang * lebar;

  writeln('Luas Persegi Panjang = ', luas);
end.
```

Output

```
tab:
idx  id          obj       type  ref  nrm  lev  adr  link
-----------------------------------------------------------
0    empty       const     1     0    0    0    0    0
1    and         const     1     0    0    0    0    0
2    array       const     1     0    0    0    0    0
3    begin       const     1     0    0    0    0    0
4    case        const     1     0    0    0    0    0
5    const       const     1     0    0    0    0    0
6    div         const     1     0    0    0    0    0
7    downto      const     1     0    0    0    0    0
8    do          const     1     0    0    0    0    0
9    else        const     1     0    0    0    0    0
10   end         const     1     0    0    0    0    0
11   for         const     1     0    0    0    0    0
12   function    const     1     0    0    0    0    0
13   if          const     1     0    0    0    0    0
14   mod         const     1     0    0    0    0    0
15   not         const     1     0    0    0    0    0
16   of          const     1     0    0    0    0    0
17   or          const     1     0    0    0    0    0
18   procedure   const     1     0    0    0    0    0
19   program     program   1     0    0    0    0    0
20   record      const     1     0    0    0    0    0
21   repeat      const     1     0    0    0    0    0
22   integer     const     1     0    0    0    0    0
23   real        const     1     0    0    0    0    0
24   boolean     const     1     0    0    0    0    0
25   char        const     1     0    0    0    0    0
26   string      const     1     0    0    0    0    0
27   then        const     1     0    0    0    0    0
28   to          const     1     0    0    0    0    0
```

```
29   type        const     1     0    0    0    0    0
30   until       const     1     0    0    0    0    0
31   var         const     1     0    0    0    0    0
32   while       const     1     0    0    0    0    0
33   integer     type      2     0    0    0    0    0
34   real        type      3     0    0    0    0    33
35   char        type      4     0    0    0    0    34
36   boolean     type      5     0    0    0    0    35
37   string      type      6     0    0    0    0    36
38   true        const     5     0    1    0    1    37
39   false       const     5     0    1    0    0    38
40   writeln     procedure 1     0    0    0    0    39
41   readln      procedure 1     0    0    0    0    40
42   HitungLuas  procedure 1     0    0    0    0    41
43   panjang     variable  2     0    1    0    0    42
44   lebar       variable  2     0    1    0    0    43
45   luas        variable  2     0    1    0    0    44

btab:
idx  last  lpar  psze  vsze
----------------------------
0    45    0     0     3    <- global block (program), vsze=3
1    45    0     0     0    <- main block (kosong variabel lokal)

atab: (kosong karena tidak ada array)

Decorated AST:
ProgramNode(name: 'HitungLuas')
 ├─ Declarations
 │    ├─ VarDecl('panjang')        → tab_index:43, type:unknown,
lev:0
 │    ├─ VarDecl('lebar')          → tab_index:44, type:unknown,
lev:0
 │    └─ VarDecl('luas')           → tab_index:45, type:unknown,
lev:0
 └─ Block                  → block_index:42, lev:0
     ├─ Assign('panjang' := 15)    → type:void
     │    ├─ target 'panjang'       → tab_index:42, type:integer
     │    └─ value 15          → type:integer
     ├─ Assign('lebar' := 8) → type:void
     │    ├─ target 'lebar'         → tab_index:43, type:integer
     │    └─ value 8           → type:integer
```

```
    ├─ Assign('luas' := panjang * lebar)      → type:void
    │   ├─ target 'luas'          → tab_index:44, type:integer
    │   └─ value BinOp 'times'          → type:integer
    │       ├─ 'panjang'          → tab_index:42, type:integer
    │       └─ 'lebar'            → tab_index:43, type:integer
    └─ writeln(...)         → predefined, tab_index:-1
        ├─ arg 'Luas Persegi Panjang = '            → type:string
        └─ arg 'luas'       → tab_index:44, type:integer
```

Screenshot Output

test > milestone-3 > output-2.txt

```
tab:
idx  id          obj       type  ref  nrm  lev  adr  link
-------------------------------------------------------
0    empty       const     1     0    0    0    0    0
1    and         const     1     0    0    0    0    0
2    array       const     1     0    0    0    0    0
3    begin       const     1     0    0    0    0    0
4    case        const     1     0    0    0    0    0
5    const       const     1     0    0    0    0    0
6    div         const     1     0    0    0    0    0
7    downto      const     1     0    0    0    0    0
8    do          const     1     0    0    0    0    0
9    else        const     1     0    0    0    0    0
10   end         const     1     0    0    0    0    0
11   for         const     1     0    0    0    0    0
12   function    const     1     0    0    0    0    0
13   if          const     1     0    0    0    0    0
14   mod         const     1     0    0    0    0    0
15   not         const     1     0    0    0    0    0
16   of          const     1     0    0    0    0    0
17   or          const     1     0    0    0    0    0
18   procedure   const     1     0    0    0    0    0
19   program     program   1     0    0    0    0    0
20   record      const     1     0    0    0    0    0
21   repeat      const     1     0    0    0    0    0
22   integer     const     1     0    0    0    0    0
23   real        const     1     0    0    0    0    0
24   boolean     const     1     0    0    0    0    0
25   char        const     1     0    0    0    0    0
26   string      const     1     0    0    0    0    0
27   then        const     1     0    0    0    0    0
28   to          const     1     0    0    0    0    0
29   type        const     1     0    0    0    0    0
30   until       const     1     0    0    0    0    0
31   var         const     1     0    0    0    0    0
32   while       const     1     0    0    0    0    0
33   integer     type      2     0    0    0    0    0
34   real        type      3     0    0    0    0    33
35   char        type      4     0    0    0    0    34
```

test > milestone-3 > output-2.txt

```
39   35   char        type      4     0    0    0    0    34
40   36   boolean     type      5     0    0    0    0    35
41   37   string      type      6     0    0    0    0    36
42   38   true        const     5     0    1    0    1    37
43   39   false       const     5     0    1    0    0    38
44   40   writeln     procedure 1     0    0    0    0    39
45   41   readln      procedure 1     0    0    0    0    40
46   42   HitungLuas  procedure 1     0    0    0    0    41
47   43   panjang     variable  2     0    1    0    0    42
48   44   lebar       variable  2     0    1    0    0    43
49   45   luas        variable  2     0    1    0    0    44
50
51   btab:
52   idx  last  lpar  psze  vsze
53   --------------------------
54   0    45    0     0     3    <- global block (program), vsze=3
55   1    45    0     0     0    <- main block (kosong variabel lokal)
56
57   atab: (kosong karena tidak ada array)
58
59   Decorated AST:
60   ProgramNode(name: 'HitungLuas')
61    ├─ Declarations
62    │    ├─ VarDecl('panjang')         → tab_index:43, type:unknown, lev:0
63    │    ├─ VarDecl('lebar')       → tab_index:44, type:unknown, lev:0
64    │    └─ VarDecl('luas')        → tab_index:45, type:unknown, lev:0
65    └─ Block                  → block_index:42, lev:0
66        ├─ Assign('panjang' := 15)     → type:void
67        │    ├─ target 'panjang'        → tab_index:42, type:integer
68        │    └─ value 15        → type:integer
69        ├─ Assign('lebar' := 8)    → type:void
70        │    ├─ target 'lebar'      → tab_index:43, type:integer
71        │    └─ value 8         → type:integer
72        ├─ Assign('luas' := panjang * lebar)   → type:void
73        │    ├─ target 'luas'       → tab_index:44, type:integer
74        │    └─ value BinOp 'times'         → type:integer
75        │         ├─ 'panjang'       → tab_index:42, type:integer
76        │         └─ 'lebar'         → tab_index:43, type:integer
77        └─ writeln(...)        → predefined, tab_index:-1
```

test > milestone-3 > output-2.txt

```
60   ProgramNode(name: 'HitungLuas')
65    └─ Block                  → block_index:42, lev:0
77        └─ writeln(...)        → predefined, tab_index:-1
78             ├─ arg 'Luas Persegi Panjang = '       → type:string
79             └─ arg 'luas'      → tab_index:44, type:integer
```

3

Input

```
program CekKelulusan;

var
  nilai: integer;
  lulus: boolean;

begin
  nilai := 75;

  if nilai >= 70 then
  begin
    lulus := true;
    writeln('Selamat, Anda Lulus!');
  end
  else
  begin
    lulus := false;
    writeln('Maaf, Anda belum lulus.');
  end;
end.
```

Output

```
tab:
idx  id          obj       type  ref  nrm  lev  adr  link
-------------------------------------------------------
0    empty       const     1     0    0    0    0    0
1    and         const     1     0    0    0    0    0
2    array       const     1     0    0    0    0    0
3    begin       const     1     0    0    0    0    0
4    case        const     1     0    0    0    0    0
5    const       const     1     0    0    0    0    0
6    div         const     1     0    0    0    0    0
7    downto      const     1     0    0    0    0    0
8    do          const     1     0    0    0    0    0
9    else        const     1     0    0    0    0    0
10   end         const     1     0    0    0    0    0
11   for         const     1     0    0    0    0    0
12   function    const     1     0    0    0    0    0
13   if          const     1     0    0    0    0    0
14   mod         const     1     0    0    0    0    0
```

```
15   not          const     1     0     0     0     0     0
16   of           const     1     0     0     0     0     0
17   or           const     1     0     0     0     0     0
18   procedure    const     1     0     0     0     0     0
19   program      program   1     0     0     0     0     0
20   record       const     1     0     0     0     0     0
21   repeat       const     1     0     0     0     0     0
22   integer      const     1     0     0     0     0     0
23   real         const     1     0     0     0     0     0
24   boolean      const     1     0     0     0     0     0
25   char         const     1     0     0     0     0     0
26   string       const     1     0     0     0     0     0
27   then         const     1     0     0     0     0     0
28   to           const     1     0     0     0     0     0
29   type         const     1     0     0     0     0     0
30   until        const     1     0     0     0     0     0
31   var          const     1     0     0     0     0     0
32   while        const     1     0     0     0     0     0
33   integer      type      2     0     0     0     0     0
34   real         type      3     0     0     0     0     33
35   char         type      4     0     0     0     0     34
36   boolean      type      5     0     0     0     0     35
37   string       type      6     0     0     0     0     36
38   true         const     5     0     1     0     1     37
39   false        const     5     0     1     0     0     38
40   writeln      procedure 1     0     0     0     0     39
41   readln       procedure 1     0     0     0     0     40
42   HitungLuas   procedure 1     0     0     0     0     41
43   panjang      variable  2     0     1     0     0     42
44   lebar        variable  2     0     1     0     0     43
45   luas         variable  2     0     1     0     0     44

btab:
idx  last  lpar  psze  vsze
----------------------------
0    45    0     0     3    <- global block (program), vsze=3
1    45    0     0     0    <- main block (kosong variabel lokal)

atab: (kosong karena tidak ada array)

Decorated AST:
ProgramNode(name: 'HitungLuas')
```

```
 ├─ Declarations
 │    ├─ VarDecl('panjang')        → tab_index:43, type:unknown,
lev:0
 │    ├─ VarDecl('lebar')          → tab_index:44, type:unknown,
lev:0
 │    └─ VarDecl('luas')           → tab_index:45, type:unknown,
lev:0
 └─ Block                  → block_index:42, lev:0
     ├─ Assign('panjang' := 15)    → type:void
     │    ├─ target 'panjang'      → tab_index:42, type:integer
     │    └─ value 15         → type:integer
     ├─ Assign('lebar' := 8) → type:void
     │    ├─ target 'lebar'        → tab_index:43, type:integer
     │    └─ value 8          → type:integer
     ├─ Assign('luas' := panjang * lebar)      → type:void
     │    ├─ target 'luas'         → tab_index:44, type:integer
     │    └─ value BinOp 'times'          → type:integer
     │        ├─ 'panjang'         → tab_index:42, type:integer
     │        └─ 'lebar'           → tab_index:43, type:integer
     └─ writeln(...)          → predefined, tab_index:-1
         ├─ arg 'Luas Persegi Panjang = '            → type:string
         └─ arg 'luas'       → tab_index:44, type:integer
```

Screenshot Output

test > milestone-3 > output-3.txt

```
tab:
idx  id          obj       type  ref  nrm  lev  adr  link
-------------------------------------------------------
0    empty       const     1     0    0    0    0    0
1    and         const     1     0    0    0    0    0
2    array       const     1     0    0    0    0    0
3    begin       const     1     0    0    0    0    0
4    case        const     1     0    0    0    0    0
5    const       const     1     0    0    0    0    0
6    div         const     1     0    0    0    0    0
7    downto      const     1     0    0    0    0    0
8    do          const     1     0    0    0    0    0
9    else        const     1     0    0    0    0    0
10   end         const     1     0    0    0    0    0
11   for         const     1     0    0    0    0    0
12   function    const     1     0    0    0    0    0
13   if          const     1     0    0    0    0    0
14   mod         const     1     0    0    0    0    0
15   not         const     1     0    0    0    0    0
16   of          const     1     0    0    0    0    0
17   or          const     1     0    0    0    0    0
18   procedure   const     1     0    0    0    0    0
19   program     program   1     0    0    0    0    0
20   record      const     1     0    0    0    0    0
21   repeat      const     1     0    0    0    0    0
22   integer     const     1     0    0    0    0    0
23   real        const     1     0    0    0    0    0
24   boolean     const     1     0    0    0    0    0
25   char        const     1     0    0    0    0    0
26   string      const     1     0    0    0    0    0
27   then        const     1     0    0    0    0    0
28   to          const     1     0    0    0    0    0
29   type        const     1     0    0    0    0    0
30   until       const     1     0    0    0    0    0
31   var         const     1     0    0    0    0    0
32   while       const     1     0    0    0    0    0
33   integer     type      2     0    0    0    0    0
34   real        type      3     0    0    0    0    33
35   char        type      4     0    0    0    0    34
```

test > milestone-3 > output-3.txt

```
39   35   char        type      4     0    0    0    0    34
40   36   boolean     type      5     0    0    0    0    35
41   37   string      type      6     0    0    0    0    36
42   38   true        const     5     0    1    0    1    37
43   39   false       const     5     0    1    0    0    38
44   40   writeln     procedure 1     0    0    0    0    39
45   41   readln      procedure 1     0    0    0    0    40
46   42   CekKelulusanprocedure 1     0    0    0    0    41
47   43   nilai       variable  2     0    1    0    0    42
48   44   lulus       variable  5     0    1    0    0    43
49
50   btab:
51   idx  last  lpar  psze  vsze
52   ---------------------------
53   0    44    0     0     2   <- global block (program), vsze=2
54   1    44    0     0     0   <- main block (kosong variabel lokal)
55   2    44    0     0     0
56   3    44    0     0     0
57
58   atab: (kosong karena tidak ada array)
59
60   Decorated AST:
61   ProgramNode(name: 'CekKelulusan')
62    ├─ Declarations
63    │    └─ VarDecl('nilai')       → tab_index:43, type:unknown, lev:0
64    │    └─ VarDecl('lulus')       → tab_index:44, type:unknown, lev:0
65    └─ Block                  → block_index:42, lev:0
66        ├─ Assign('nilai' := 75)   → type:void
67        │   ├─ target 'nilai'      → tab_index:42, type:integer
68        │   └─ value 75        → type:integer
69        └─ IfStmt                → type:void
70            ├─ condition BinOp 'geq'       → type:boolean
71            │   ├─ 'nilai'         → tab_index:42, type:integer
72            │   └─ 70      → type:integer
73            ├─ then CompoundStmt
74            │   ├─ Assign('lulus' := true)     → type:void
75            │   │   ├─ target 'lulus'      → tab_index:43, type:boolean
76            │   │   └─ value 'true'        → tab_index:37, type:boolean
77            │   └─ writeln(...)        → predefined, tab index:-1
```

```
test > milestone-3 > output-3.txt
61   ProgramNode(name: 'CekKelulusan')
65    └─ Block                  → block_index:42, lev:0
69        └─ IfStmt               → type:void
77   │    │   └─ writeln(...)        → predefined, tab_index:-1
78   │    │       └─ arg 'Selamat, Anda Lulus!'      → type:string
79   │    └─ else CompoundStmt
80   │        ├─ Assign('lulus' := false)    → type:void
81   │        │   ├─ target 'lulus'      → tab_index:43, type:boolean
82   │        │   └─ value 'false'       → tab_index:38, type:boolean
83   │        └─ writeln(...)        → predefined, tab_index:-1
84   │            └─ arg 'Maaf, Anda belum lulus.'       → type:string
```

4

Input

```
<program>
├──<program-header>
│  ├──programsy
│  ├──ident (HitungExposure)
│  └──semicolon
├──<declaration-part>
│  └──<var-declaration>
│     ├──varsy
│     ├──<identifier-list>
│     │  ├──ident (iso)
│     │  ├──comma
│     │  └──ident (shutter)
│     ├──colon
│     ├──<type>
│     │  └──ident (integer)
│     ├──semicolon
│     ├──<identifier-list>
│     │  └──ident (aperture)
│     ├──colon
│     ├──<type>
│     │  └──ident (real)
│     └──semicolon
├──<compound-statement>
│  ├──beginsy
│  ├──<statement-list>
│  │  ├──<statement>
│  │  │  └──<assignment-statement>
│  │  │     ├──<variable>
│  │  │     │  └──ident (iso)
```


becomes
<expression>
<simple-expression>
<term>
<factor>
intcon (400)
semicolon
<statement>
<assignment-statement>
<variable>
ident (shutter)
becomes
<expression>
<simple-expression>
<term>
<factor>
intcon (250)
semicolon
<statement>
<assignment-statement>
<variable>
ident (aperture)
becomes
<expression>
<simple-expression>
<term>
<factor>
realcon (2.8)
semicolon
<statement>
<if-statement>
ifsy
<expression>
<simple-expression>
<term>
<factor>
lparent
<expression>
<simple-expression>
<term>
<factor>
ident (iso)

```
| | | |   |   | | ├──<relational-operator>
| | | |   |   | | |  └──geq
| | | |   |   | |  └──<simple-expression>
| | | |   |   | |   └──<term>
| | | |   |   | |    └──<factor>
| | | |   |   | |     └──intcon (800)
| | | |   |   |  └──rparent
| | | |   |   ├──<multiplicative-operator>
| | | |   |   |  └──andsy
| | | |   |   └──<factor>
| | | |   |    ├──lparent
| | | |   |    ├──<expression>
| | | |   |    | ├──<simple-expression>
| | | |   |    | | └──<term>
| | | |   |    | |  └──<factor>
| | | |   |    | |   └──ident (aperture)
| | | |   |    | ├──<relational-operator>
| | | |   |    | |  └──leq
| | | |   |    | └──<simple-expression>
| | | |   |    |  └──<term>
| | | |   |    |   └──<factor>
| | | |   |    |    └──realcon (1.4)
| | | |   |    └──rparent
| | | |   ├──thensy
| | | |   └──<statement>
| | | |    └──<procedure/function-call>
| | | |     ├──ident (writeln)
| | | |     ├──lparent
| | | |     ├──<parameter-list>
| | | |     | └──<expression>
| | | |     |  └──<simple-expression>
| | | |     |   └──<term>
| | | |     |    └──<factor>
| | | |     |     └──string ('Foto overexposed')
| | | |     └──rparent
| | ├──semicolon
| | └──<statement>
| └──endsy
└──period
```

Output

```
tab:
```

```
idx  id          obj       type  ref  nrm  lev  adr  link
-------------------------------------------------------
0    empty       const     1     0    0    0    0    0
1    and         const     1     0    0    0    0    0
2    array       const     1     0    0    0    0    0
3    begin       const     1     0    0    0    0    0
4    case        const     1     0    0    0    0    0
5    const       const     1     0    0    0    0    0
6    div         const     1     0    0    0    0    0
7    downto      const     1     0    0    0    0    0
8    do          const     1     0    0    0    0    0
9    else        const     1     0    0    0    0    0
10   end         const     1     0    0    0    0    0
11   for         const     1     0    0    0    0    0
12   function    const     1     0    0    0    0    0
13   if          const     1     0    0    0    0    0
14   mod         const     1     0    0    0    0    0
15   not         const     1     0    0    0    0    0
16   of          const     1     0    0    0    0    0
17   or          const     1     0    0    0    0    0
18   procedure   const     1     0    0    0    0    0
19   program     program   1     0    0    0    0    0
20   record      const     1     0    0    0    0    0
21   repeat      const     1     0    0    0    0    0
22   integer     const     1     0    0    0    0    0
23   real        const     1     0    0    0    0    0
24   boolean     const     1     0    0    0    0    0
25   char        const     1     0    0    0    0    0
26   string      const     1     0    0    0    0    0
27   then        const     1     0    0    0    0    0
28   to          const     1     0    0    0    0    0
29   type        const     1     0    0    0    0    0
30   until       const     1     0    0    0    0    0
31   var         const     1     0    0    0    0    0
32   while       const     1     0    0    0    0    0
33   integer     type      2     0    0    0    0    0
34   real        type      3     0    0    0    0    33
35   char        type      4     0    0    0    0    34
36   boolean     type      5     0    0    0    0    35
37   string      type      6     0    0    0    0    36
38   true        const     5     0    1    0    1    37
39   false       const     5     0    1    0    0    38
```

```
40   writeln     procedure 1     0    0    0    0    39
41   readln      procedure 1     0    0    0    0    40
42   HitungExposureprocedure 1     0    0    0    0    41
43   iso         variable  2     0    1    0    0    42
44   shutter     variable  2     0    1    0    0    43
45   aperture    variable  3     0    1    0    0    44

btab:
idx  last  lpar  psze  vsze
---------------------------
0    45    0     0     3   <- global block (program), vsze=3
1    45    0     0     0   <- main block (kosong variabel lokal)

atab: (kosong karena tidak ada array)

Decorated AST:
ProgramNode(name: 'HitungExposure')
 ├─ Declarations
 │    ├─ VarDecl('iso')            → tab_index:43, type:unknown,
lev:0
 │    └─ VarDecl('shutter')        → tab_index:44, type:unknown,
lev:0
 │    └─ VarDecl('aperture')       → tab_index:45, type:unknown,
lev:0
 └─ Block                  → block_index:42, lev:0
     ├─ Assign('iso' := 400) → type:void
     │    ├─ target 'iso'           → tab_index:42, type:integer
     │    └─ value 400         → type:integer
     ├─ Assign('shutter' := 250)   → type:void
     │    ├─ target 'shutter'       → tab_index:43, type:integer
     │    └─ value 250         → type:integer
     ├─ Assign('aperture' := 2.8)  → type:void
     │    ├─ target 'aperture'            → tab_index:44, type:real
     │    └─ value 2.8         → type:real
     └─ IfStmt                  → type:void
         ├─ condition BinOp 'andsy'            → type:boolean
         │    ├─ BinOp 'geq'         → type:boolean
         │    │    ├─ 'iso'          → tab_index:42, type:integer
         │    │    └─ 800            → type:integer
         │    └─ BinOp 'leq'         → type:boolean
         │         ├─ 'aperture'           → tab_index:44, type:real
         │         └─ 1.4            → type:real
         │
```

```
└─ then writeln(...)                    → predefined, tab_index:-1
   └─ arg 'Foto overexposed'                  → type:string
```

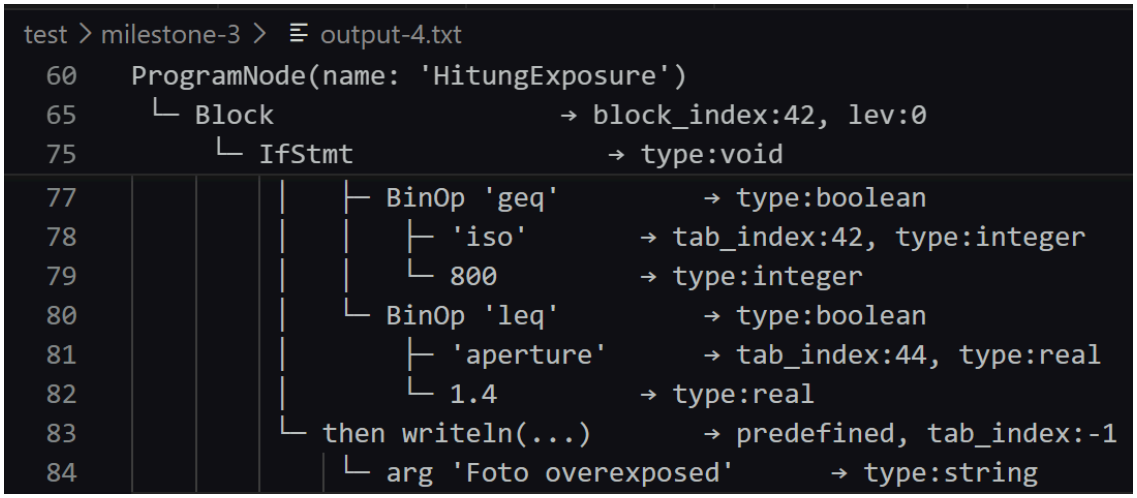
Screenshot Output

test > milestone-3 > output-4.txt

```
tab:
idx  id          obj       type  ref  nrm  lev  adr  link
-------------------------------------------------------
0    empty       const     1     0    0    0    0    0
1    and         const     1     0    0    0    0    0
2    array       const     1     0    0    0    0    0
3    begin       const     1     0    0    0    0    0
4    case        const     1     0    0    0    0    0
5    const       const     1     0    0    0    0    0
6    div         const     1     0    0    0    0    0
7    downto      const     1     0    0    0    0    0
8    do          const     1     0    0    0    0    0
9    else        const     1     0    0    0    0    0
10   end         const     1     0    0    0    0    0
11   for         const     1     0    0    0    0    0
12   function    const     1     0    0    0    0    0
13   if          const     1     0    0    0    0    0
14   mod         const     1     0    0    0    0    0
15   not         const     1     0    0    0    0    0
16   of          const     1     0    0    0    0    0
17   or          const     1     0    0    0    0    0
18   procedure   const     1     0    0    0    0    0
19   program     program   1     0    0    0    0    0
20   record      const     1     0    0    0    0    0
21   repeat      const     1     0    0    0    0    0
22   integer     const     1     0    0    0    0    0
23   real        const     1     0    0    0    0    0
24   boolean     const     1     0    0    0    0    0
25   char        const     1     0    0    0    0    0
26   string      const     1     0    0    0    0    0
27   then        const     1     0    0    0    0    0
28   to          const     1     0    0    0    0    0
29   type        const     1     0    0    0    0    0
30   until       const     1     0    0    0    0    0
31   var         const     1     0    0    0    0    0
32   while       const     1     0    0    0    0    0
33   integer     type      2     0    0    0    0    0
34   real        type      3     0    0    0    0    33
35   char        type      4     0    0    0    0    34
```

test > milestone-3 > output-4.txt

```
39   35   char        type      4     0    0    0    0    34
40   36   boolean     type      5     0    0    0    0    35
41   37   string      type      6     0    0    0    0    36
42   38   true        const     5     0    1    0    1    37
43   39   false       const     5     0    1    0    0    38
44   40   writeln     procedure 1     0    0    0    0    39
45   41   readln      procedure 1     0    0    0    0    40
46   42   HitungExposureprocedure 1     0    0    0    0    41
47   43   iso         variable  2     0    1    0    0    42
48   44   shutter     variable  2     0    1    0    0    43
49   45   aperture    variable  3     0    1    0    0    44
50
51   btab:
52   idx  last  lpar  psze  vsze
53   ---------------------------
54   0    45    0     0     3   <- global block (program), vsze=3
55   1    45    0     0     0   <- main block (kosong variabel lokal)
56
57   atab: (kosong karena tidak ada array)
58
59   Decorated AST:
60   ProgramNode(name: 'HitungExposure')
61     ├─ Declarations
62     │    ├─ VarDecl('iso')         → tab_index:43, type:unknown, lev:0
63     │    └─ VarDecl('shutter')         → tab_index:44, type:unknown, lev:0
64     │    └─ VarDecl('aperture')        → tab_index:45, type:unknown, lev:0
65     └─ Block                  → block_index:42, lev:0
66        ├─ Assign('iso' := 400)    → type:void
67        │   ├─ target 'iso'        → tab_index:42, type:integer
68        │   └─ value 400       → type:integer
69        ├─ Assign('shutter' := 250)    → type:void
70        │   ├─ target 'shutter'        → tab_index:43, type:integer
71        │   └─ value 250       → type:integer
72        ├─ Assign('aperture' := 2.8)   → type:void
73        │   ├─ target 'aperture'       → tab_index:44, type:real
74        │   └─ value 2.8       → type:real
75        └─ IfStmt                → type:void
76           ├─ condition BinOp 'andsy'         → type:boolean
77           │   ├─ BinOp 'geq'         → type:boolean
```



5	Input

```
<program>
├── <program-header>
│   ├── PROGRAMSY(program)
│   ├── IDENT(Hello)
│   └── SEMICOLON(;)
├── <declaration-part>
│   └── <var-declaration>
│       ├── VARSY(var)
│       ├── <identifier-list>
│       │   ├── IDENT(a)
│       │   ├── COMMA(,)
│       │   └── IDENT(b)
│       ├── COLON(:)
│       ├── <type>
│       │   └── IDENT(integer)
│       └── SEMICOLON(;)
├── <compound-statement>
│   ├── BEGINSY(begin)
│   ├── <statement-list>
│   │   ├── <assignment-statement>
│   │   │   ├── IDENT(a)
│   │   │   ├── BECOMES(:=)
│   │   │   └── <expression>
│   │   │       └── <simple-expression>
│   │   │           └── <term>
│   │   │               └── <factor>
│   │   │                   └── INTCON(5)
```

```
│   │   ├── SEMICOLON(;)
│   │   ├── <assignment-statement>
│   │   │   ├── IDENT(b)
│   │   │   ├── BECOMES(:=)
│   │   │   └── <expression>
│   │   │       ├── <simple-expression>
│   │   │       │   ├── <term>
│   │   │       │   │   └── <factor>
│   │   │       │   │       └── IDENT(a)
│   │   │       │   ├── <additive-operator>
│   │   │       │   │   └── PLUS(+)
│   │   │       │   └── <term>
│   │   │       │       └── <factor>
│   │   │       │           └── NUMBER(10)
│   │   ├── SEMICOLON(;)
│   │   ├── <procedure/function-call>
│   │   │   ├── IDENT(writeln)
│   │   │   ├── LPARENT(()
│   │   │   ├── <parameter-list>
│   │   │   │   ├── <expression>
│   │   │   │   │   └── <simple-expression>
│   │   │   │   │       └── <term>
│   │   │   │   │           └── <factor>
│   │   │   │   │               └── STRING('Result = ')
│   │   │   │   ├── COMMA(,)
│   │   │   │   └── <expression>
│   │   │   │       └── <simple-expression>
│   │   │   │           └── <term>
│   │   │   │               └── <factor>
│   │   │   │                   └── IDENT(b)
│   │   │   ├── RPARENT())
│   │   │   └── SEMICOLON(;)
│   └── ENDSY(end)
└── PERIOD(.)
```

Output

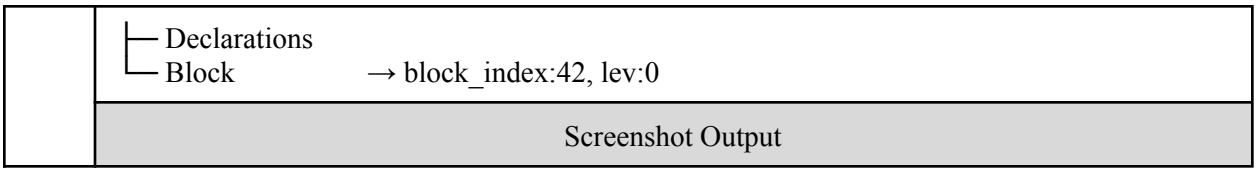
```
tab:
idx  id         obj      type  ref  nrm  lev  adr  link
-------------------------------------------------------
0    empty      const    1     0    0    0    0    0
1    and        const    1     0    0    0    0    0
2    array      const    1     0    0    0    0    0
3    begin      const    1     0    0    0    0    0
```

```
4   case       const    1    0    0    0    0    0
5   const      const    1    0    0    0    0    0
6   div        const    1    0    0    0    0    0
7   downto     const    1    0    0    0    0    0
8   do         const    1    0    0    0    0    0
9   else       const    1    0    0    0    0    0
10  end        const    1    0    0    0    0    0
11  for        const    1    0    0    0    0    0
12  function   const    1    0    0    0    0    0
13  if         const    1    0    0    0    0    0
14  mod        const    1    0    0    0    0    0
15  not        const    1    0    0    0    0    0
16  of         const    1    0    0    0    0    0
17  or         const    1    0    0    0    0    0
18  procedure  const    1    0    0    0    0    0
19  program    program  1    0    0    0    0    0
20  record     const    1    0    0    0    0    0
21  repeat     const    1    0    0    0    0    0
22  integer    const    1    0    0    0    0    0
23  real       const    1    0    0    0    0    0
24  boolean    const    1    0    0    0    0    0
25  char       const    1    0    0    0    0    0
26  string     const    1    0    0    0    0    0
27  then       const    1    0    0    0    0    0
28  to         const    1    0    0    0    0    0
29  type       const    1    0    0    0    0    0
30  until      const    1    0    0    0    0    0
31  var        const    1    0    0    0    0    0
32  while      const    1    0    0    0    0    0
33  integer    type     2    0    0    0    0    0
34  real       type     3    0    0    0    0   33
35  char       type     4    0    0    0    0   34
36  boolean    type     5    0    0    0    0   35
37  string     type     6    0    0    0    0   36
38  true       const    5    0    1    0    1   37
39  false      const    5    0    1    0    0   38
40  writeln    procedure 1   0    0    0    0   39
41  readln     procedure 1   0    0    0    0   40
42  Hello      program  1    0    0    0    0   41

btab:
idx  last  lpar  psze  vsze
---------------------------
0   42    0     0     0  <- global block (program), vsze=0
1   42    0     0     0  <- main block (kosong variabel lokal)

atab: (kosong karena tidak ada array)

Decorated AST:
ProgramNode(name: 'Hello')
```


Declarations
Block → block_index:42, lev:0
Screenshot Output

test > milestone-3 > output-5.txt

```
tab:
idx  id          obj       type  ref  nrm  lev  adr  link
-------------------------------------------------------
0    empty       const     1     0    0    0    0    0
1    and         const     1     0    0    0    0    0
2    array       const     1     0    0    0    0    0
3    begin       const     1     0    0    0    0    0
4    case        const     1     0    0    0    0    0
5    const       const     1     0    0    0    0    0
6    div         const     1     0    0    0    0    0
7    downto      const     1     0    0    0    0    0
8    do          const     1     0    0    0    0    0
9    else        const     1     0    0    0    0    0
10   end         const     1     0    0    0    0    0
11   for         const     1     0    0    0    0    0
12   function    const     1     0    0    0    0    0
13   if          const     1     0    0    0    0    0
14   mod         const     1     0    0    0    0    0
15   not         const     1     0    0    0    0    0
16   of          const     1     0    0    0    0    0
17   or          const     1     0    0    0    0    0
18   procedure   const     1     0    0    0    0    0
19   program     program   1     0    0    0    0    0
20   record      const     1     0    0    0    0    0
21   repeat      const     1     0    0    0    0    0
22   integer     const     1     0    0    0    0    0
23   real        const     1     0    0    0    0    0
24   boolean     const     1     0    0    0    0    0
25   char        const     1     0    0    0    0    0
26   string      const     1     0    0    0    0    0
27   then        const     1     0    0    0    0    0
28   to          const     1     0    0    0    0    0
29   type        const     1     0    0    0    0    0
30   until       const     1     0    0    0    0    0
31   var         const     1     0    0    0    0    0
32   while       const     1     0    0    0    0    0
33   integer     type      2     0    0    0    0    0
34   real        type      3     0    0    0    0    33
35   char        type      4     0    0    0    0    34
```

test > milestone-3 > output-5.txt

```
39   35   char        type      4     0     0     0     0     34
40   36   boolean     type      5     0     0     0     0     35
41   37   string      type      6     0     0     0     0     36
42   38   true        const     5     0     1     0     1     37
43   39   false       const     5     0     1     0     0     38
44   40   writeln     procedure 1     0     0     0     0     39
45   41   readln      procedure 1     0     0     0     0     40
46   42   Hello       program   1     0     0     0     0     41
47
48   btab:
49   idx  last  lpar  psze  vsze
50   ---------------------------
51   0    42    0     0     0    <- global block (program), vsze=0
52   1    42    0     0     0    <- main block (kosong variabel lokal)
53
54   atab: (kosong karena tidak ada array)
55
56   Decorated AST:
57   ProgramNode(name: 'Hello')
58   ├─ Declarations
59   └─ Block                  → block_index:42, lev:0
```

6

Input

```
program UjiArray;

var
  nilai: array [1..5] of integer;
  huruf: array [0..9] of char;
  i, sum: integer;

begin
  sum := 0;

  for i := 1 to 5 do
  begin
    nilai[i] := i * 10;
    sum := sum + nilai[i];
  end;

  huruf[0] := 'A';
  huruf[1] := 'B';
```

```
  writeln('Total sum array = ', sum);
end.
```

Output

```
tab:
idx  id          obj       type  ref  nrm  lev  adr  link
-------------------------------------------------------
0    empty       const     1     0    0    0    0    0
1    and         const     1     0    0    0    0    0
2    array       const     1     0    0    0    0    0
3    begin       const     1     0    0    0    0    0
4    case        const     1     0    0    0    0    0
5    const       const     1     0    0    0    0    0
6    div         const     1     0    0    0    0    0
7    downto      const     1     0    0    0    0    0
8    do          const     1     0    0    0    0    0
9    else        const     1     0    0    0    0    0
10   end         const     1     0    0    0    0    0
11   for         const     1     0    0    0    0    0
12   function    const     1     0    0    0    0    0
13   if          const     1     0    0    0    0    0
14   mod         const     1     0    0    0    0    0
15   not         const     1     0    0    0    0    0
16   of          const     1     0    0    0    0    0
17   or          const     1     0    0    0    0    0
18   procedure   const     1     0    0    0    0    0
19   program     program   1     0    0    0    0    0
20   record      const     1     0    0    0    0    0
21   repeat      const     1     0    0    0    0    0
22   integer     const     1     0    0    0    0    0
23   real        const     1     0    0    0    0    0
24   boolean     const     1     0    0    0    0    0
25   char        const     1     0    0    0    0    0
26   string      const     1     0    0    0    0    0
27   then        const     1     0    0    0    0    0
28   to          const     1     0    0    0    0    0
29   type        const     1     0    0    0    0    0
30   until       const     1     0    0    0    0    0
31   var         const     1     0    0    0    0    0
32   while       const     1     0    0    0    0    0
33   integer     type      2     0    0    0    0    0
```

```
34   real        type      3     0    0    0    0    33
35   char        type      4     0    0    0    0    34
36   boolean     type      5     0    0    0    0    35
37   string      type      6     0    0    0    0    36
38   true        const     5     0    1    0    1    37
39   false       const     5     0    1    0    0    38
40   writeln     procedure 1     0    0    0    0    39
41   readln      procedure 1     0    0    0    0    40
42   UjiArray    procedure 1     0    0    0    0    41
43   nilai       variable  7     0    1    0    0    42
44   huruf       variable  7     1    1    0    0    43
45   i           variable  2     0    1    0    0    44
46   sum         variable  2     0    1    0    0    45

btab:
idx  last  lpar  psze  vsze
----------------------------
0    46    0     0     4    <- global block (program), vsze=4
1    46    0     0     0    <- main block (kosong variabel lokal)

atab:
idx  xtyp  etyp  eref  low  high  elsz  size
-----------------------------------------------
0    2     2     0     1    5     1     5
1    2     4     0     0    9     1     10

Decorated AST:
ProgramNode(name: 'UjiArray')
 ├─ Declarations
 │    └─ VarDecl('nilai')          → tab_index:43, type:unknown,
lev:0
 │    └─ VarDecl('huruf')          → tab_index:44, type:unknown,
lev:0
 │    ├─ VarDecl('i')        → tab_index:45, type:unknown, lev:0
 │    └─ VarDecl('sum')            → tab_index:46, type:unknown,
lev:0
 └─ Block                  → block_index:42, lev:0
     ├─ Assign('sum' := 0)   → type:void
     │    ├─ target 'sum'           → tab_index:45, type:integer
     │    └─ value 0          → type:integer
     ├─ ForStmt(iter: 'i')  → type:void
     │    ├─ start 1          → type:integer
```

```
│   ├─ end 5                → type:integer
├─ Assign('huruf[0]' := 'A')  → type:void
│   ├─ target 'huruf'        → tab_index:43, type:char
│   │   └─ index 0           → type:integer
│   └─ value 'A'             → type:char
├─ Assign('huruf[1]' := 'B')  → type:void
│   ├─ target 'huruf'        → tab_index:43, type:char
│   │   └─ index 1           → type:integer
│   └─ value 'B'             → type:char
└─ writeln(...)              → predefined, tab_index:-1
    ├─ arg 'Total sum array = '          → type:string
    └─ arg 'sum'             → tab_index:45, type:integer
```

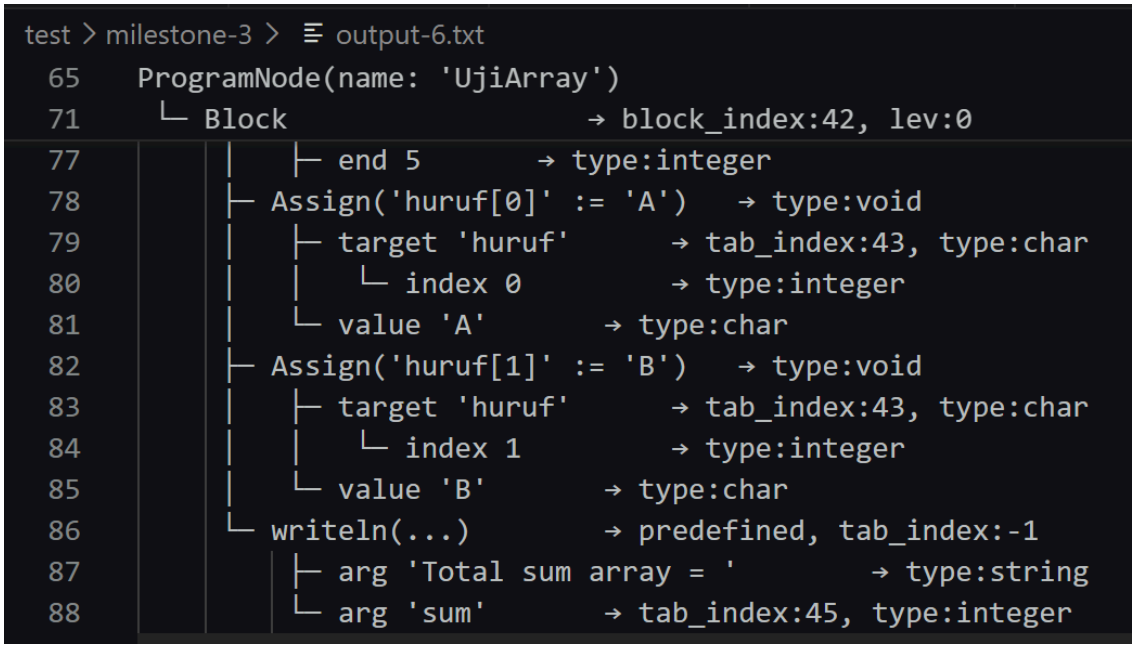
Screenshot Output

test > milestone-3 > output-6.txt

```
tab:
idx  id          obj       type  ref  nrm  lev  adr  link
-------------------------------------------------------
0    empty       const     1     0    0    0    0    0
1    and         const     1     0    0    0    0    0
2    array       const     1     0    0    0    0    0
3    begin       const     1     0    0    0    0    0
4    case        const     1     0    0    0    0    0
5    const       const     1     0    0    0    0    0
6    div         const     1     0    0    0    0    0
7    downto      const     1     0    0    0    0    0
8    do          const     1     0    0    0    0    0
9    else        const     1     0    0    0    0    0
10   end         const     1     0    0    0    0    0
11   for         const     1     0    0    0    0    0
12   function    const     1     0    0    0    0    0
13   if          const     1     0    0    0    0    0
14   mod         const     1     0    0    0    0    0
15   not         const     1     0    0    0    0    0
16   of          const     1     0    0    0    0    0
17   or          const     1     0    0    0    0    0
18   procedure   const     1     0    0    0    0    0
19   program     program   1     0    0    0    0    0
20   record      const     1     0    0    0    0    0
21   repeat      const     1     0    0    0    0    0
22   integer     const     1     0    0    0    0    0
23   real        const     1     0    0    0    0    0
24   boolean     const     1     0    0    0    0    0
25   char        const     1     0    0    0    0    0
26   string      const     1     0    0    0    0    0
27   then        const     1     0    0    0    0    0
28   to          const     1     0    0    0    0    0
29   type        const     1     0    0    0    0    0
30   until       const     1     0    0    0    0    0
31   var         const     1     0    0    0    0    0
32   while       const     1     0    0    0    0    0
33   integer     type      2     0    0    0    0    0
34   real        type      3     0    0    0    0    33
35   char        type      4     0    0    0    0    34
```

test > milestone-3 > output-6.txt

```
39   35   char        type      4     0    0    0    0    34
40   36   boolean     type      5     0    0    0    0    35
41   37   string      type      6     0    0    0    0    36
42   38   true        const     5     0    1    0    1    37
43   39   false       const     5     0    1    0    0    38
44   40   writeln     procedure 1     0    0    0    0    39
45   41   readln      procedure 1     0    0    0    0    40
46   42   UjiArray    procedure 1     0    0    0    0    41
47   43   nilai       variable  7     0    1    0    0    42
48   44   huruf       variable  7     1    1    0    0    43
49   45   i           variable  2     0    1    0    0    44
50   46   sum         variable  2     0    1    0    0    45
51
52   btab:
53   idx  last  lpar  psze  vsze
54   ---------------------------
55   0    46    0     0     4   <- global block (program), vsze=4
56   1    46    0     0     0   <- main block (kosong variabel lokal)
57
58   atab:
59   idx  xtyp  etyp  eref  low  high  elsz  size
60   ------------------------------------------------
61   0    2     2     0     1    5     1     5
62   1    2     4     0     0    9     1     10
63
64   Decorated AST:
65   ProgramNode(name: 'UjiArray')
66     ├─ Declarations
67     │    └─ VarDecl('nilai')       → tab_index:43, type:unknown, lev:0
68     │    └─ VarDecl('huruf')       → tab_index:44, type:unknown, lev:0
69     │    ├─ VarDecl('i')       → tab_index:45, type:unknown, lev:0
70     │    └─ VarDecl('sum')         → tab_index:46, type:unknown, lev:0
71     └─ Block                 → block_index:42, lev:0
72         ├─ Assign('sum' := 0)  → type:void
73         │   ├─ target 'sum'        → tab_index:45, type:integer
74         │   └─ value 0         → type:integer
75         ├─ ForStmt(iter: 'i')  → type:void
76         │   ├─ start 1         → type:integer
77         │   ├─ end 5       → type:integer
```



4. Kesimpulan Dan Saran

4.1 Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan pada Milestone 3, dapat disimpulkan bahwa program Semantic Analyzer untuk Arion Compiler telah berhasil diimplementasikan secara modular menggunakan bahasa C++ dengan menerapkan prinsip pemrograman berorientasi objek (OOP). Sistem ini berhasil mereduksi Parse Tree menjadi Abstract Syntax Tree (AST) melalui proses Syntax-Directed Translation yang membuang token terminal tak bermakna untuk evaluasi semantik, seperti keyword begin, end, dan tanda titik koma. Pengelolaan ruang lingkup (scope) dan deklarasi identifier juga telah sukses diimplementasikan melalui integrasi tiga tabel simbol utama, yaitu tab untuk identifier utama, atab untuk informasi array, dan btab untuk mengelola block scope hierarki subprogram. Selanjutnya, proses dekorasi AST (Type & Scope Checking) berjalan dengan baik menggunakan pendekatan Visitor Pattern (L-Attributed Grammar) secara Depth-First Search (DFS). Program juga telah berhasil menerapkan aturan validasi semantik (Type Compatibility) bahasa Arion secara tepat, mencakup kewajiban kesamaan tipe data pada assignment dengan dukungan widening conversion (seperti nilai Integer ke penampung Real), serta evaluasi kondisi wajib bertipe Boolean pada instruksi if dan while.

4.2 Saran

a. Mekanisme penanganan pesan error (seperti type mismatch, undeclared identifier, atau out-of-bounds array) dapat dibuat lebih informatif dengan menyertakan detail spesifik berupa

nomor baris (line number) dan posisi kolom terjadinya error di source code, agar pengguna lebih mudah melakukan debugging.

b. Mengingat pembangunan AST dan Symbol Table banyak menggunakan pembuatan struktur objek secara dinamis (pointer/tree node), disarankan untuk menggunakan manajemen memori modern (seperti smart pointers) untuk memastikan tidak terjadi memory leak saat compiler memproses kode sumber berskala besar.

c. Pemeriksaan semantik dapat diperluas tidak hanya untuk mendeteksi error (kesalahan fatal), tetapi juga menampilkan peringatan atau warnings, misalnya mendeteksi variabel yang telah dideklarasikan namun tidak pernah digunakan (unused variables) di dalam tabel simbol, atau blok kode yang tidak mungkin dieksekusi (unreachable code)

5. Lampiran

Github: https://github.com/Rmadhian/DGN-Tubes-IF2224-2026

Tabel Kontribusi:

NIM	Kontribusi	Persentase
13523150	Mengimplementasi semantic_stmt.cpp & laporan	33,33%
13524117	-	0%
13524120	Mengimplementasi semantic_decl.cpp, ast_builder & laporan	33,34%
13524126	Mengimplementasi semantic_expr.cpp, semantic_printer.cpp & laporan	33,33%

6. Referensi

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Addison-Wesley.

Cooper, K. D., & Torczon, L. (2011). *Engineering a Compiler* (2nd ed.). Morgan Kaufmann.

Wirth, N. (1976). *Algorithms + Data Structures = Programs*. Prentice-Hall.